

1 ans

1.1

So, if there are n people in the group. An if n is even Each member toss a coin and there is a probability of head or tail, only 2 cases. Suppose, if the first person got the head, for this scenario every other person will either say the same or different. Consider all are heads, so the count of same will be even. If we count of number of different cases that will always be even if the manager has paid. Every time the count of different will increase by 2.

1.2

With the same scenario as above, we will consider the number of different will always be even, if n is even. But, in case if one of the friend has paid then they will lie at the table, which makes it clear that the number of different will not be even (It will increase or decrease by 1). This is clear that the manager hasn't paid and they don't want to know which friend has paid. So, it can stop here. Similarly, if n is odd, the different tosses will be odd. And If manager has paid for the meal, it will be even.

1.3

Each person tosses with all the other. So, number of tosses for each would be $n-1$. For, n people the number of tosses is $n(n-1)$. $N(n-1)$ will be even value always. This means the the different or same value cases will be even if the manager has paid. And it would be odd if one of friend has paid. Because, one of the person would lie.

2 ans.

1. So, for the first dice a outcome is generated and let it be 'a' in between 1 to 6. And similarly, some other value 'b' for next roll of dice.
If $a+b$ is less than or equal to 12, go back to $a+b$. Otherwise if $a+b$ is greater than 12, we go back to first step.

This algorithm can generate the desired value, because the sum k is the value upto 12. And probability of sum k is probability of generating the two numbers which sum is k

- In this algorithm we can see and prove that the sum k is generating values of the sum of the values of two dices. Because the probability of sum k returning is the value generating two numbers which sum is less than or equal to 12.
- So, in order to prove this algorithm is giving correct probabilities, we have to get that the probability of returning a sum k value is equal to probability of generating two numbers whose sum is k .
- We know that a and b are simply random numbers between 1 to 6. The probability of sum k is just the probability of k divided by the probability of generating the number k less than or equal to 12.
- Therefore, its proved.

2.(a) If n is the power of 2
we have to generate a random output form 0 to $n-1$.

We have a randomly generate a number between 0 to $n-1$ and let be 'a'

If a is odd then we return to a. Otherwise, if a is even we go back to first step.

- We can see that this algorithm generates a random output. So, the chance of getting the random k is probability of generating the value k, divided by the probability of generating a number less than n.
- To prove, it generates a random sample and then we have to show that the probability of returning a number k is equal to probability of generating a number k.
- Since a is generated uniformly at randomly, probability of generating the number k is the probability of generating a number k divided by the probability of generating a number less than n
- So, probability of returning a number k is equal to probability of generating a number k and algorithm generates a random sample.
- Therefore, if n is the power of 2 we can use this algorithm.

2.(b) if n is not the power of 2.

Again we have to generate like for the above problem. Generate a value between 0 to $n-1$.

If a is odd, then return to 1. Otherwise, if a is even we go back to starting step..

- If in case this method fails to return a value k then start from step 1.
- This algorithm generates a random sample uniformly. The probability of returning the number k is the probability if generating the value k, divided by the probability of generating a number less than n.
- The chances of this being correct is $\frac{1}{2}$ of k, if k is number of trials. So, the expected number of trials has to be 2 to generate a random output.
- To prove, we need to show that the probability of returning k should be probability of generating k.
- So, as a is a random output, the probability of generating k is same as probability of generating number k divided by probability of generating number less than n.
- So, if n is not power of 2 this above algorithm can be used to generate random sample output.

3ans:

So, we have n children in a row and we can swap any 2 and rearrange. We can use only n swaps to reorder them In a random uniform order.

- For each children i in the row, we assign it as k_i . And this k_i is in the range of 1 to n.
- Now, according to their k_i values we arrange them in the ascending order in the row.
- For each children I, swap it with its next position $(i+1)$.
- K_i values will be generated randomly, so, the children will be arranged randomly.
- And also each child is swapped with its next child, they will be arranged in uniformly random order.
- This sorting take place in linear sorting of swaps. Sorting of elements take $O(n \log n)$. The running time of this will be $O(n \log n)$.
- Proof, this algorithm generates random order and assigns that to children by creating random integers.
- And, we swap each child with other next so it arranges in the randomly uniform order.

4Ans

4.1.

first, we have to collect all the tickets in the big bowl.

- select one ticket randomly and discard the remaining
- repeat the process until we have only one remaining and then that one ticket is the winner

Proof: this algorithm takes the uniformly random ticket. We need to prove that each ticket has the same probability to select. We can do this by induction.

So, in the first stage each ticket has same probability for selection. At some middle m th stage, each ticket has the same probability to select and we have to show that $m+1$ th stage also.

So, at $m+1$ th stage the ticket is not selected yet. So, all the tickets have the same probability to choose, and each ticket is very random and has equal chances. Therefore, at each and every step, each ticket has the same probability of being chosen. Therefore, this algorithm chooses a uniformly random ticket.

4.2

Like the above one we follow the similar algorithm.

- We have to keep all the tickets in the bowl.
- Randomly select the k tickets and remove the rest.
- Repeat until k tickets remain in the bowl.

Proof: for this proof also we have to show that the probability of selecting each ticket is the same. We can do this by induction. In the first stage each ticket has the same probability of selection. And In the middle stage(induction stage) also each ticket has the same probability to select. So $m+1$ th stage also should have the same probability for each ticket to be chosen.

So, in $m+1$ th stage tickets are not yet chosen but still each ticket have the same chances of being selected. Each ticket is selected randomly so in all of the tickets, each one have the same chances to select. Therefore, this algorithm provides uniformly random output.